

AI技术应用验证报告

姓名：王涵

学号：2023141461082

一、验证场景与目标

为验证前文所设计的“软件过程改进方案”的可行性，我选取了“编码实现与测试验证”这一核心改进场景开展最小可行验证（MVP）。

- 验证场景：**Spring Boot 后端应用中复杂业务逻辑（用户权限校验与动态路由菜单生成）的代码编写，以及配套单元测试代码的生成。
- 验证目标：**评估真实AI工具在“代码生成”与“单测生成”任务上的准确率、开发耗时，并对比传统纯人工方式，输出可量化的效率与质量差异数据。

二、工具选型与测试环境

- 选用AI工具：**GitHub Copilot (集成于 IntelliJ IDEA) + ChatGPT-4o (用于复杂逻辑推理规划)。
- 测试对象：**项目中的 RolePermissionService 模块。
- 实验变量：**
 - 对照组（人工组）：**模拟过往未使用AI的开发过程，通过查阅API、手敲代码和手动编写 JUnit 5用例完成。
 - 实验组（AI辅助组）：**使用自然语言写好注释，引导Copilot生成业务代码，并通过ChatGPT-4o一键生成覆盖异常分支的单元测试。

三、执行过程记录

1. AI辅助代码生成执行过程

- 步骤一（上下文构建）：**在IDE中创建 SysMenu 和 SysRole 的实体类。
- 步骤二（Prompt引导）：**在 SysMenuServiceImpl 类中编写自然语言注释：
// 根据用户的角色ID列表，查询所有启用的菜单，构建成树形结构并按照sortOrder排序，最后缓存到Redis中，设置1小时过期。
- 步骤三（AI生成）：**回车后，GitHub Copilot 瞬间读取了上下文中的实体类字段，自动生成了完整的由Stream API构成的递归建树代码，并正确引入了 RedisTemplate。
- 步骤四（人工调整）：**AI生成的代码总体正确，但Redis的Key命名略显不规范，人工花费了2分钟微调了Key的前缀常量。

测试分支覆盖率对比 (%)

传统人工: 50%

AI 辅助 : 88%

五、 结论与改进建议

验证结论

通过本次最小可行验证（MVP）证明，将生成式AI技术（Copilot、LLM）引入到代码编写与单元测试环节，不仅是“理论可行”，而是能产生**立竿见影的数倍效率提升**。AI擅长处理样板代码、复杂的流处理API转换以及耗时的Mock环境搭建，从而让开发人员能够专注于系统设计和业务架构本身。

改进落地建议

- 统一工具集与账号分配：**为团队所有开发人员配置统一的GitHub Copilot企业版账号，避免使用不同的免费工具导致代码风格不统一。
- 建立团队Prompt沉淀库：**AI的输出质量高度依赖于提示词（Prompt）质量。团队应建立共享文档，沉淀“如何让AI生成符合公司规范代码”的标准提示词模板。
- 强化AI代码的Review红线：**验证中发现AI偶尔会生成不符合团队命名习惯的代码。因此，必须在过程中强制规定：无论AI生成的代码跑得多顺，“**未经人工逐行审查并理解的AI代码，严禁合并提交**”。